RAPPORT DE PROJET

Déploiement Automatisé d’une Infrastructure Cloud

avec Terraform sur AWS

Application Flask | EC2 | S3 | RDS PostgreSQL

Ibrahim

Mars 2026

Table des matières

1. Introduction
2. Architecture du projet
3. Préparation de l’environnement
4. Déploiement de l’infrastructure
5. Vérification sur la console AWS
6. Tests de l’application
7. Destruction de l’infrastructure
8. Problèmes rencontrés et solutions
9. Conclusion

1. Introduction

L'objectif de ce projet est de déployer une infrastructure cloud sur AWS de manière automatisée avec Terraform. Concrètement, il s'agit de mettre en place une application web Flask avec un stockage S3 pour les fichiers et une base de données PostgreSQL, le tout sans avoir à configurer quoi que ce soit manuellement sur la console AWS.

Le scénario est celui d'un ingénieur DevOps qui doit déployer une application avec un espace de stockage pour les images, les logs et d'autres fichiers. Terraform permet de tout automatiser en décrivant l'infrastructure dans des fichiers de configuration.

2. Architecture du projet

Voici les différents composants que j'ai déployés sur AWS :

- Un VPC avec des subnets publics et privés pour séparer les ressources
- Une instance EC2 sous Ubuntu 22.04 avec Flask, Gunicorn et Nginx
- Un bucket S3 pour stocker les fichiers (images, logs, uploads)
- Une base de données RDS PostgreSQL pour les métadonnées
- Un rôle IAM pour que la VM puisse accéder au S3
- Des Security Groups pour gérer les accès réseau

Côté fichiers Terraform, j'ai organisé le code en plusieurs fichiers : provider.tf pour la config AWS, variables.tf pour les variables, main.tf pour les ressources, outputs.tf pour les infos de sortie, et userdata.sh.tpl pour installer automatiquement tout ce qu'il faut sur la VM.

3. Préparation de l'environnement

Avant de commencer, j'ai installé Terraform et AWS CLI sur mon Mac avec Homebrew. J'ai aussi généré une clé SSH pour pouvoir me connecter à la VM plus tard. Ensuite j'ai configuré mes clés d'accès AWS avec aws configure en choisissant la région eu-west-3 (Paris).

Les informations sensibles comme les clés AWS et le mot de passe de la base de données sont dans un fichier terraform.tfvars qui n'est pas envoyé sur Git grâce au .gitignore.

4. Déploiement de l'infrastructure

4.1 Initialisation

J'ai commencé par lancer terraform init pour télécharger le provider AWS et préparer le projet.

```
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
Ibra@MacBook-Air-de-Salima terraform %
```

Figure 1 : terraform init — Initialisation réussie

4.2 Planification

Ensuite, terraform plan m'a montré tout ce qui allait être créé. Au total, 23 ressources étaient prévues : le VPC, les subnets, les security groups, l'EC2, le S3, la RDS, etc.

```
Plan: 23 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + app_url            = (known after apply)
  + deployment_summary = (known after apply)
  + ec2_public_dns     = (known after apply)
  + ec2_public_ip      = (known after apply)
  + flask_api_url      = (known after apply)
  + rds_endpoint       = (known after apply)
  + rds_hostname       = (known after apply)
  + s3_bucket_arn      = (known after apply)
  + s3_bucket_name     = (known after apply)
  + s3_bucket_region   = (known after apply)
  + ssh_command        = (known after apply)
```

Figure 2 : terraform plan — 23 ressources à créer

4.3 Application

J'ai lancé terraform apply pour créer toutes les ressources. Ça a pris environ 8 minutes, surtout à cause de la base RDS qui est longue à créer. À la fin, Terraform affiche les outputs avec l'IP de la VM, le nom du bucket S3, l'endpoint de la base, etc.

```
EOT
ec2_public_dns = "ec2-15-237-250-200.eu-west-3.compute.amazonaws.com"
ec2_public_ip = "15.237.250.200"
flask_api_url = "http://15.237.250.200:5000"
rds_endpoint = "flask-cloud-app-db.c9cyisw6c3k5.eu-west-3.rds.amazonaws.com:5432"
rds_hostname = "flask-cloud-app-db.c9cyisw6c3k5.eu-west-3.rds.amazonaws.com"
s3_bucket_arn = "arn:aws:s3:::flask-app-static-files-ibrahim-2025"
s3_bucket_name = "flask-app-static-files-ibrahim-2025"
s3_bucket_region = "eu-west-3"
ssh_command = "ssh -i ~/.ssh/id_rsa ubuntu@15.237.250.200"
Ibra@MacBook-Air-de-Salima terraform %
```

Figure 3 : Outputs de terraform apply — IP, S3 et RDS

La VM a reçu l’IP 15.237.250.200, le bucket S3 s’appelle flask-app-static-files-ibrahim-2025, et la base RDS est accessible depuis la VM via son endpoint.

5. Vérification sur la console AWS

5.1 Instance EC2

Sur la console EC2, on voit que l'instance flask-cloud-app est bien lancée et passe les 3 vérifications de santé.

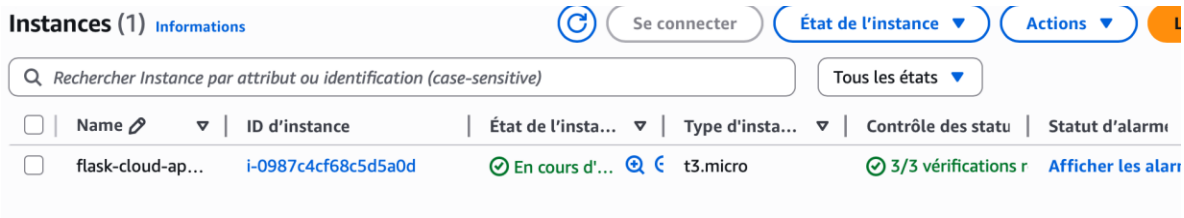

Figure 4 : Console EC2 — Instance en cours d'exécution

5.2 Bucket S3

Le bucket contient bien les 3 dossiers prévus : images/, logs/ et uploads/. Le versionning est activé et l'accès public est bloqué.

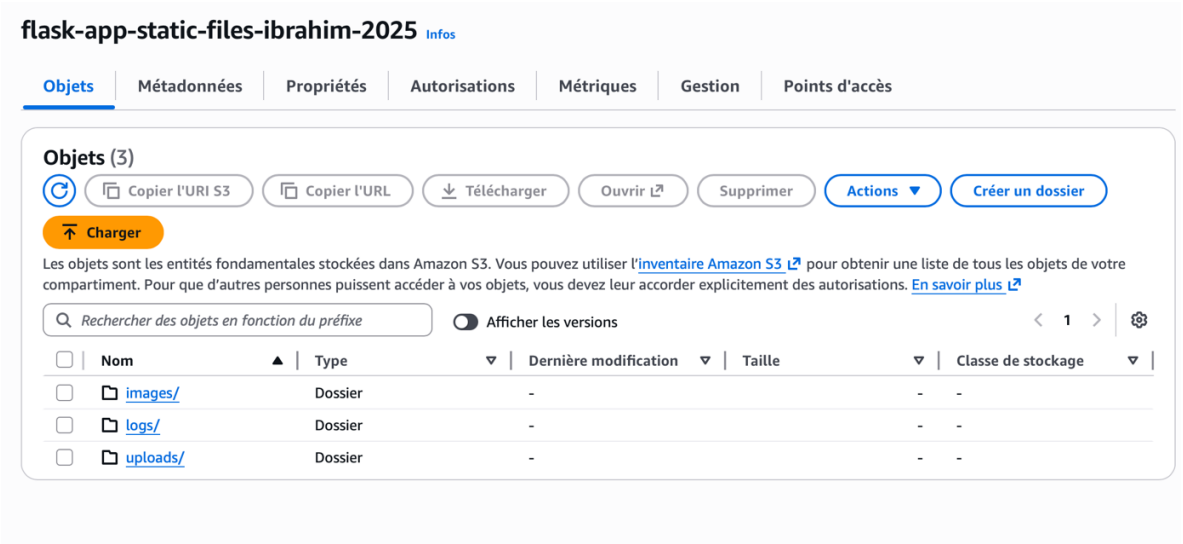

Figure 5 : Console S3 — Dossiers images/, logs/, uploads/

5.3 Base de données RDS

La base PostgreSQL est disponible dans la région eu-west-3. Elle n'est accessible que depuis l'EC2 grâce au Security Group.

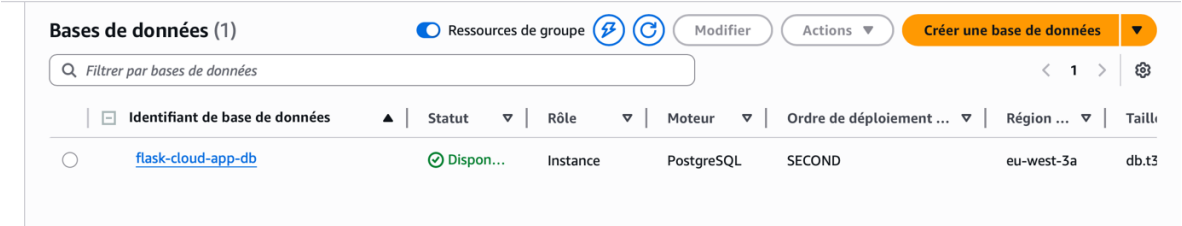

Figure 6 : Console RDS — Base PostgreSQL disponible

6. Tests de l’application

Après avoir attendu quelques minutes que Flask s’installe sur la VM, j’ai testé l’API avec curl.

6.1 Health Check

Le health check confirme que l’application, le S3 et la base de données fonctionnent bien. Tout est à "ok".

```
Ibra@MacBook-Air-de-Salima terraform % curl -s http://15.237.250.200/api/health

{"app":"ok","database":"ok","s3":"ok",
mestamp":"2026-03-12T12:40:02.442183"}
Ibra@MacBook-Air-de-Salima terraform % C
```

Figure 7 : Health check — Tout fonctionne

6.2 Upload de fichier

J’ai uploadé un fichier test.txt via l’API. Le fichier est envoyé dans le bucket S3 et ses infos sont enregistrées en base.

```
{"file":{"category":"uploads","created_at":"2026-03-12T12:41:28.113762","description":"Test upload","file_size":0,"file_type":"text/plain","filename":"test.txt","id":"
fdd885d-db41-4ea0-9ffb-99b439d16819","s3_key":"uploads/0fdd885d-db41-4ea0-9ffb-99b439d16819_test.txt","updated_at":"2026-03-12T12:41:28.113765"},"message":"Fichier upl
ad\u00e9 avec succ\u00e8s"}
Ibra@MacBook-Air-de-Salima terraform %
```

Figure 8 : Upload d’un fichier — Réussi

6.3 Liste des fichiers

La route GET /api/files renvoie bien le fichier qu’on vient d’uploader avec toutes ses métadonnées.

```
{"count":1,"files":[{"category":"uploads","created_at":"2026-03-12T12:41:28.113762","description":"Test upload","file_size":0,"file_type":"text/plain","filename":"tes
txt","id":"0fdd885d-db41-4ea0-9ffb-99b439d16819","s3_key":"uploads/0fdd885d-db41-4ea0-9ffb-99b439d16819_test.txt","updated_at":"2026-03-12T12:41:28.113765"}]}
Ibra@MacBook-Air-de-Salima terraform %
```

Figure 9 : Liste des fichiers en base

6.4 Liste des objets S3

En interrogeant directement le bucket S3, on retrouve les dossiers et le fichier uploadé.

```
Ibra@MacBook-Air-de-Salima terraform % curl -s http://15.237.250.200/api/s3/list

{"bucket":"flask-app-static-files-ibrahim-2025","objects":[{"key":"images/","last_modified":"2026-03-12T1
3-12T12:23:33+00:00","size":0},{"key":"uploads/","last_modified":"2026-03-12T12:23:33+00:00","size":0},{"
ast_modified":"2026-03-12T12:41:29+00:00","size":16}],"prefix":""}
Ibra@MacBook-Air-de-Salima terraform %
```

Unable to handle /System/Library/Frameworks/Python.framework/...

Figure 10 : Objets dans le bucket S3

6.5 Suppression

La suppression fonctionne aussi : le fichier est retiré du S3 et de la base.

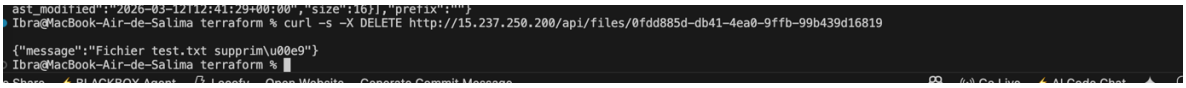

Figure 11 : Suppression du fichier — OK

7. Destruction de l'infrastructure

Une fois les tests terminés, j'ai lancé terraform destroy pour tout supprimer. Les 23 ressources ont été détruites correctement. C'est important de le faire pour éviter d'être facturé par AWS.

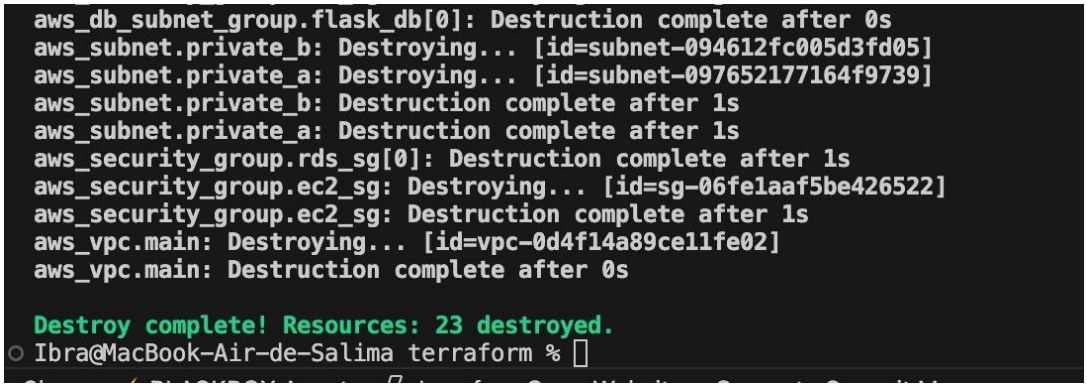

Figure 12 : terraform destroy — 23 ressources détruites

8. Problèmes rencontrés et solutions

8.1 Erreur dans terraform.tfvars

Au début, j'avais mis les valeurs sans guillemets dans terraform.tfvars, ce qui provoquait une erreur. En Terraform, les chaînes doivent être entre guillemets doubles. Une fois corrigé, ça marchait.

8.2 Erreur dans le script userdata

Le script userdata.sh.tpl avait un commentaire avec ${...} que Terraform essayait d'interpréter comme une variable. J'ai juste retiré cette syntaxe du commentaire et c'était réglé.

8.3 Application pas accessible tout de suite

Après terraform apply, l'app n'était pas accessible immédiatement. Il faut attendre 2-3 minutes que le script d'installation finisse sur la VM. On peut vérifier en SSH avec le fichier /var/log/userdata.log.

9. Conclusion

Ce projet m’a permis de comprendre comment fonctionne Terraform et l’Infrastructure as Code. J’ai pu déployer une infrastructure complète sur AWS (réseau, VM, stockage, base de données) avec quelques fichiers de configuration.

L’application Flask déployée gère un CRUD complet pour les fichiers, avec le stockage dans S3 et les métadonnées dans PostgreSQL. Tout est automatisé : du réseau jusqu’au lancement de l’app.

Le gros avantage de cette approche, c’est qu’on peut recréer toute l’infra avec un seul terraform apply, et tout supprimer avec terraform destroy. C’est rapide, reproductible, et on évite les erreurs de configuration manuelle.